GROW21

Project submitted to the

School of Engineering & Technology, BML Munjal University

for the partial fulfilment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and Engineering

Submitted by

Kartikey Bishnoi (240506) **Bhavya Bansal (240427)**

Dhruva Jhanjhari (240446) **Om Dixit (240492)**

BML MUNJAL UNIVERSITY

FROM HERE TO THE WORLD

Under the Guidance of

Dr. Nishtha Phutela

School of Engineering and Technology

BML Munjal University

Kapriwas, Haryana – 122413

May 5, 2026

Certificate

Date: 4/29/2026

This is to certify that the work presented in this project titled Grow21 has been carried out by Kartikey Bishnoi, Dhruva Jhanjhari, Bhavya, and Om Dixit under my supervision.

The work is genuine, original, and suitable for submission to BML Munjal University for the award of Bachelor of Technology in School of Engineering and Technology.

_____

Supervisor

Dr. Nishtha Phutela

Designation:

Department of Computer Science and Engineering

BML Munjal University, Kapriwas, Haryana

Table of Contents

Sl. No.	Topic	Page No.
1	Abstract	1
2	Abbreviations	1
3	List of Tables	2
4	List of Figures	2
5	Introduction	3
6	Problem Statement	3
7	Methodology	3
8	ER Diagram	6
9	Schema Design & Normalization	9
10	SQL Implementation	10
11	Discussion / Results	13
12	Output Screens	14
13	Concluding Remarks	19
14	Societal Relevance	19
15	Future Scope	19
16	Team Details & Contribution	20

Abstract

Grow21 is a Database Management System designed to support the learning and development of children with Trisomy 21 (Down Syndrome). The system aims to make learning more structured, simple, and personalized according to each child's needs, focusing on improving key skills such as motor skills, vocabulary, and basic understanding through organized learning modules.

It provides a unified platform for teachers, parents, and administrators to manage student information such as login details, profiles, and learning records. The system integrates with a mobile application, from which students' learning progress and activity data are fetched and stored in the database. This allows users to easily track each child's progress from initial assessment to regular updates, helping identify strengths and areas that need improvement.

The system is built using relational database concepts and follows Third Normal Form (3NF) to ensure data consistency and reduce redundancy. It also supports advanced queries for generating meaningful reports and insights.

In this case study, Grow21 demonstrates the practical application of RDBMS concepts such as the entity-relationship model, normalization, primary and foreign keys, indexes, triggers, and the use of scalar and aggregate functions along with views. The solution is developed using Node.js, MySQL, HTML, CSS, and JavaScript, with React for the frontend interface, while the mobile application is used to collect and update student progress data.

Overall, Grow21 creates an efficient and supportive learning environment where children can grow at their own pace while their progress is effectively monitored and guided.

Abbreviations

Short Form	Full Form
DBMS	Database Management System
ER	Entity Relationship
3NF	Third Normal Form
PK	Primary Key
FK	Foreign Key
SQL	Structured Query Language
DDL	Data Definition Language
DML	Data Manipulation Language
DQL	Data Query Language
TCL	Transaction Control Language
ACID	Atomicity, Consistency, Isolation, Durability

List of Tables

Table No.	Description
Table 1	Entities and Attributes
Table 2	Relationships Between Entities

List of Figures

Figure No.	Description
Figure 1	ER Diagrams
Figure 2	Home Page Role Selection
Figure 3	Parent Login Page
Figure 4	Parent Dashboard Child Selection
Figure 5	Parent Dashboard Child Performance
Figure 6	Teacher Dashboard Overview
Figure 7	Teacher Registered Children Page
Figure 8	Admin Dashboard Overview
Figure 9	Admin All Children
Figure 10	Search Bar to Filter by student name

Introduction

Children with Trisomy 21 require personalized learning approaches that match their cognitive and developmental abilities. Grow21 is designed to support these needs by providing a structured and digital system to manage all learning-related data in one place. It stores information about children, parents, educators, and learning activities in a well-organized relational database.

Instead of relying on scattered records or manual tracking, the system creates a centralized platform where all data is connected and easy to access. It also integrates with a mobile application, from which student progress and activity data are collected and stored. This allows consistent tracking of each child's development over multiple sessions and helps in identifying patterns in learning performance over time.

Problem Statement

In India, there are limited digital platforms specifically designed to support the learning needs of children with Trisomy 21 (Down Syndrome). Most existing solutions are either too general or do not provide structured, personalized learning and progress tracking. As a result, parents and educators often depend on manual methods or scattered digital tools, which make it difficult to consistently monitor a child's development.

There is a need for a dedicated system that not only supports learning activities but also tracks progress in a structured and meaningful way. Grow21 addresses this gap by providing a unified platform where student data, learning sessions, and progress information are systematically stored and managed.

To support this functionality, a well-designed relational database is required to ensure data consistency, reduce redundancy, and allow efficient retrieval of information. This enables better tracking of student performance, improved communication between parents and teachers, and more informed decision-making for personalized learning.

Methodology

The development of the Grow21 system followed a structured database design approach, where each step was based on the actual requirements of tracking learning progress for children with Trisomy 21.

Phase 1: Requirements Analysis

The system requirements were identified based on how teachers and parents need to track a child's learning progress. The system needed to store details of children, parents, educators, and schools, along with learning modules, lessons, sessions, and performance data.

Special focus was given to capturing session-based learning data, where each session contains multiple attempts with information such as correctness and response time. The system also needed to support login functionality and generate progress reports for each child.

Phase 2: Conceptual Design (ER Diagram)

An Entity-Relationship (ER) diagram was created to represent the structure of the system. The main entities identified include Parent, Child, Educator, School, Module, Lesson, Session, Attempt, ProgressReport, and User.

Key design decisions include:

- A Parent → Child (1:N) relationship to represent that one parent can have multiple children
- A School → Educator (1:N) relationship
- A Child ↔ Educator (M:N) relationship resolved using the *Child_Educator* table
- A Module → Lesson (1:N) structure to organize learning content
- A Session → Attempt (1:N) relationship to capture detailed performance data

This structure ensures that learning activities and progress can be tracked at multiple levels.

Phase 3: Logical Design (Relational Schema)

The ER diagram was converted into relational tables using MySQL. Each entity was mapped to a table, and relationships were maintained using primary keys and foreign keys.

For example:

- Child references ParentID
- Session links Child and Lesson
- Attempt stores detailed performance data linked to sessions
- Child_Educator is used as a bridge table to handle many-to-many relationships

This design ensures proper linkage of data across all components of the system.

Phase 4: Normalization

The database was normalized up to Third Normal Form (3NF) to remove redundancy and ensure data consistency.

- All tables have primary keys and atomic values (1NF)
- No partial dependency exists in tables with composite keys such as *Child_Educator* (2NF)
- No transitive dependency exists, as non-key attributes depend only on primary keys (3NF)

This ensures efficient storage and avoids duplication of data.

Phase 5: Implementation

The database was implemented in MySQL using DDL and DML commands. Tables such as *Child, Session, Attempt,* and *ProgressReport* were created, and sample data was inserted.

Advanced SQL features were also implemented:

- Indexes were added on foreign keys to improve query performance
- Views such as *ProgressView* and *Child_Session_Summary* were created to simplify data retrieval

- Stored procedures like *GetChildProgressReport* were used to generate dynamic progress reports
- Aggregate functions and joins were used to calculate accuracy, session counts, and performance trends

Phase 6: Application Integration

The system is integrated with both a mobile application and a web-based dashboard. The backend is developed using Node.js, while the frontend is built using React (JSX), HTML, CSS, and JavaScript, with SVG assets used for visual elements.

The mobile application is used by children to perform learning activities. As the child interacts with the app, session data and performance details such as accuracy, response time, and attempts are captured and stored in the database through the backend.

The web dashboard is used by parents, teachers, and administrators to access this data. It retrieves and displays information using SQL queries, views, and stored procedures, allowing users to monitor progress, analyze performance, and generate reports.

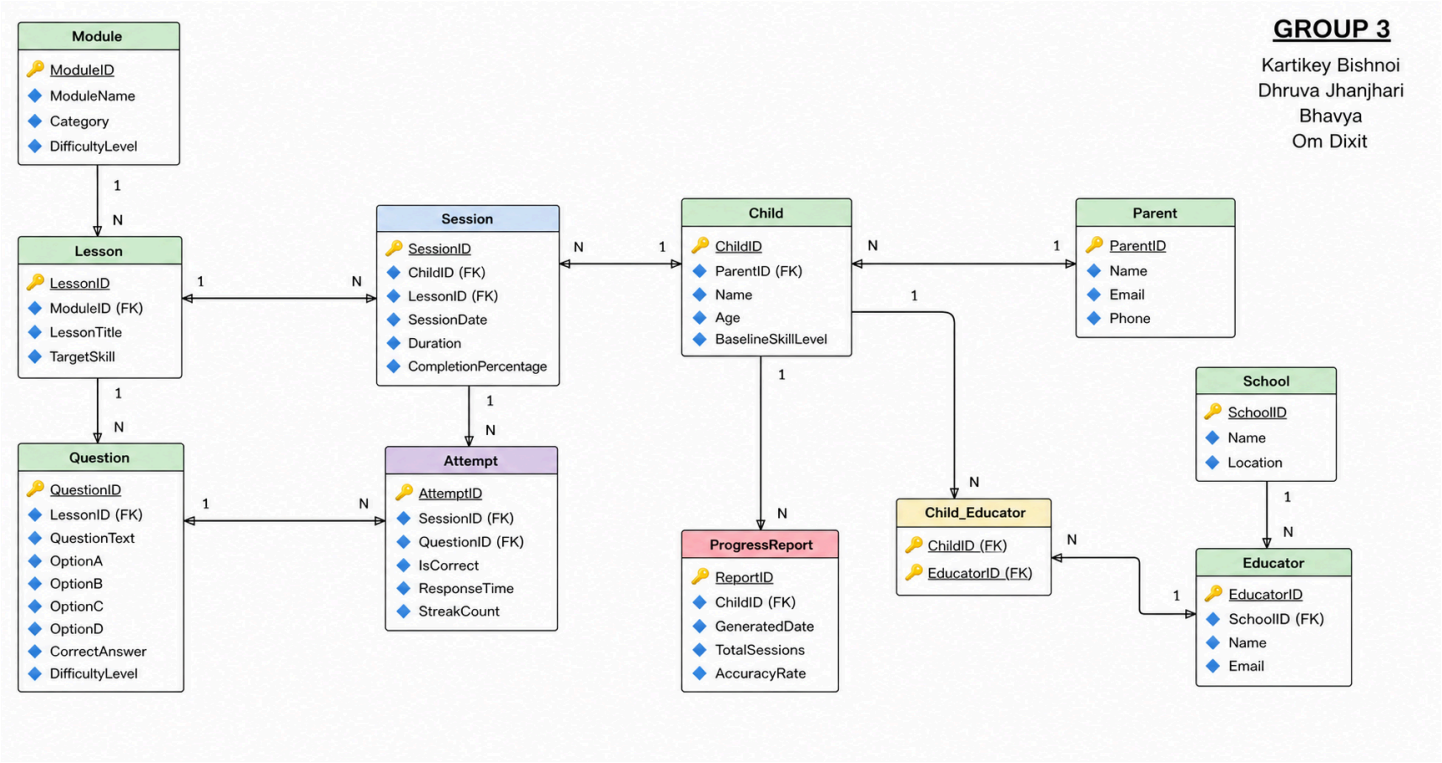
Algorithm: Student Progress Tracking Flow

Input: Child activity data generated from the mobile application (ie. Child ID, Lesson ID, Attempts, Response Time, Correct/Incorrect)

Output: Updated Progress Report (ie. Accuracy, Total Sessions, Performance Summary)

Step 1: User logs in through the application - credentials are verified using the User table.
Step 2: Parent creates or selects a child profile - stored in the Child table.
Step 3: When the child starts a learning activity on the mobile app, a new Session record is created.
Step 4: As the child answers questions, each interaction is automatically recorded in the Attempt table with details such as correctness and response time.
Step 5: Each session is linked with a Lesson and Module to identify the type of activity performed.
Step 6: SQL queries process this data to calculate performance metrics like total attempts and accuracy.
Step 7: The stored procedure GetChildProgressReport is executed to compute overall progress.
Step 8: A new record is inserted into the ProgressReport table with updated performance values.
Step 9: The dashboard retrieves and displays this data for parents and teachers using views and queries.

ER Diagram(s)



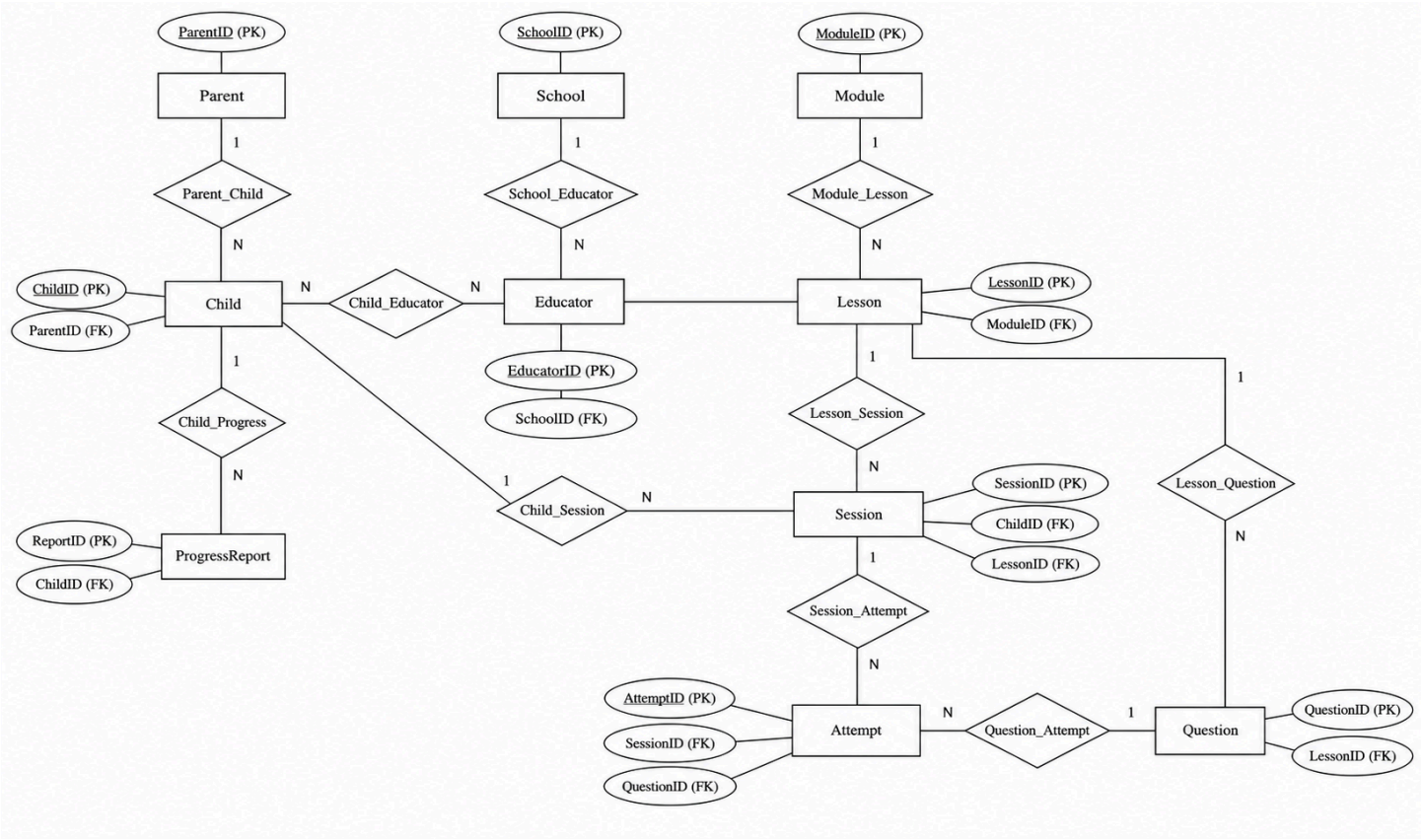


Figure 1. ER Diagram and Relational Schema

Entities and Attributes

Table 1

Entity	Primary Key	Attributes	Explanation
User	UserID	Email, Password, Role, Created_At	Stores login details for parent, teacher, and admin
Parent	ParentID	UserID (FK), Name, Email	Stores parent information linked to a user account
Child	ChildID	ParentID (FK), Name, Age, BaselineSkillLevel	Stores details of each child under a parent
School	SchoolID	Name, Location	Stores school information
Educator	EducatorID	SchoolID (FK), UserID (FK), Name, Email	Stores teacher details linked to a school and login account
Child_Educator	ChildID, EducatorID	(no extra attributes)	Connects children and teachers (many-to-many relationship)
Module	ModuleID	ModuleName, Category, DifficultyLevel	Represents learning categories (like language, math)
Lesson	LessonID	ModuleID (FK), LessonTitle, TargetSkill	Stores lessons inside each module
Question	QuestionID	LessonID (FK), QuestionText, CorrectAnswer, DifficultyLevel	Stores questions used in games
Session	SessionID	ChildID (FK), LessonID (FK), SessionDate, Duration, CompletionPercentage	Stores each gameplay session of a child
Attempt	AttemptID	SessionID (FK), QuestionID (FK), IsCorrect, ResponseTime, StreakCount	Stores each answer given by the child during a session
ProgressReport	ReportID	ChildID (FK), GeneratedDate, TotalSessions, AccuracyRate	Stores overall performance summary of a child

Relationships

• Parent → Child (1:N): One parent can have multiple children, but each child belongs to one parent.

• User → Parent (1:1): Each parent has one login account in the system.

• User → Educator (1:1): Each educator has one login account.

• School → Educator (1:N): One school can have multiple educators.

• Child ↔ Educator (M:N): A child can be assigned to multiple educators and an educator can handle multiple children (implemented using Child_Educator table).

• Module → Lesson (1:N): One module contains multiple lessons.

• Lesson → Question (1:N): Each lesson includes multiple questions.

• Child → Session (1:N): A child can participate in multiple learning sessions.

• Lesson → Session (1:N): Each session is associated with a specific lesson.

• Session → Attempt (1:N): One session contains multiple attempts (answers).

• Question → Attempt (1:N): Each attempt corresponds to one question.

• Child → ProgressReport (1:N): A child can have multiple progress reports over time.

Database Tables Summary

Table 2

Table	Purpose
User	Stores login credentials for parents, educators, and admins
Parent	Stores parent details linked to user accounts
Child	Stores child profiles under each parent
School	Stores school information
Educator	Stores teacher details linked to school and user
Child_Educator	Maps children to educators (many-to-many relationship)
Module	Defines learning categories
Lesson	Stores lessons under each module
Question	Stores questions used in learning games
Session	Tracks each gameplay session of a child
Attempt	Stores responses and performance for each question
ProgressReport	Stores overall performance summary of each child

Schema Design & Normalization

The database schema for Grow21 is designed using a relational model to support a learning application involving children, parents, educators, and game-based modules.

Each entity from the ER diagram is mapped to a separate table with a primary key (PK) for unique identification. Relationships between tables are maintained using foreign keys (FK), ensuring referential integrity and consistent data linkage.

For example, the Child table references ParentID, the Session table links Child and Lesson, and the Attempt table connects sessions with questions.

The schema includes key structures such as:

- Parent–Child (1:M) relationship to manage multiple children under one parent
- School–Educator (1:M) relationship
- Child–Educator (M:N) relationship resolved using the Child_Educator bridge table
- Module–Lesson–Question hierarchy to organize learning content
- Session–Attempt structure to track activity and performance
- ProgressReport table to store aggregated performance metrics

The database is normalized up to Third Normal Form (3NF) to eliminate redundancy and ensure data consistency. All attributes are atomic (1NF), non-key attributes fully depend on the primary key (2NF), and there are no transitive dependencies (3NF). For instance, parent details are not duplicated in the Child table but referenced through foreign keys.

Redundancy is minimized by storing each type of data in its respective table and linking them through keys rather than duplication. This improves maintainability and avoids inconsistencies.

To enhance performance, indexing is applied. Primary keys are automatically indexed, and additional indexes are created on frequently used foreign keys such as ChildID, ParentID, and SessionID, improving query execution and join efficiency.

Overall, the schema ensures data integrity, scalability, and efficient management of learning and performance data within the system.

SQL Implementation

The database for Grow21 was implemented using MySQL. Tables were created using DDL commands, and sample data was inserted using DML queries. Various SQL operations such as joins, aggregate functions, views, and stored procedures were used to manage and analyze the data efficiently.

CREATE TABLE - DDL

```
CREATE TABLE Child (
    ChildID           INT PRIMARY KEY,
    ParentID          INT NOT NULL,
    Name              VARCHAR(100) NOT NULL,
    DateOfBirth       DATE NOT NULL,
    BaselineSkillLevel ENUM('Beginner','Intermediate','Advanced') NOT NULL DEFAULT 'Beginner',
    CONSTRAINT fk_child_user
        FOREIGN KEY (ChildID) REFERENCES `User`(UserID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_child_parent
        FOREIGN KEY (ParentID) REFERENCES Parent(ParentID)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
```

DML

```
INSERT INTO Child (ChildID, ParentID, Name, DateOfBirth, BaselineSkillLevel)
VALUES (101, 2, 'Rohan', '2020-01-15', 'Beginner');
```

DQL (JOIN + GROUP BY)

```
SELECT
c.Name,
COUNT(s.SessionID) AS TotalSessions
FROM Child c
JOIN Session s
ON c.ChildID = s.ChildID
GROUP BY c.ChildID;
```

Aggregate Function

```
SELECT
s.ChildID,
ROUND(AVG(a.IsCorrect) * 100, 2) AS Accuracy
FROM Attempt a
JOIN Session s
ON a.SessionID = s.SessionID
GROUP BY s.ChildID;
```

View

```
CREATE VIEW ProgressView AS
SELECT
s.ChildID,
ROUND(AVG(a.IsCorrect) * 100, 2) AS Accuracy
FROM Session s
JOIN Attempt a
ON s.SessionID = a.SessionID
GROUP BY s.ChildID;
```

Stored Procedure

```
CALL GetChildProgressReport(1);
```

Trigger

```
CREATE TRIGGER trg_child_validate_dob
BEFORE INSERT ON Child
FOR EACH ROW
BEGIN
IF NEW.DateOfBirth >= CURDATE() THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Date of birth must be in the past';
END IF;
END;
```

TCL

```
START TRANSACTION;

INSERT INTO User (Username, Password, Role)
VALUES ('testuser', 'pass123', 'Parent');
ROLLBACK;
```

Indexing

```
CREATE INDEX idx_child_parent ON Child(ParentID);
CREATE INDEX idx_session_child ON Session(ChildID);
CREATE INDEX idx_attempt_session ON Attempt(SessionID);
```

Discussion / Results

The database worked properly and all relationships between the tables were maintained correctly using foreign keys. We were able to store and manage data for users, children, sessions, and attempts without any issues or inconsistencies.

The queries we implemented, such as calculating total sessions and accuracy, gave meaningful results which can be directly used in the application dashboard. We also used views like *ProgressView* to calculate performance dynamically, instead of storing extra or duplicate data.

Overall, the system is able to track a child's learning progress based on their activity in the mobile application in a clear and structured way, making it easier for parents and teachers to monitor improvement over time.

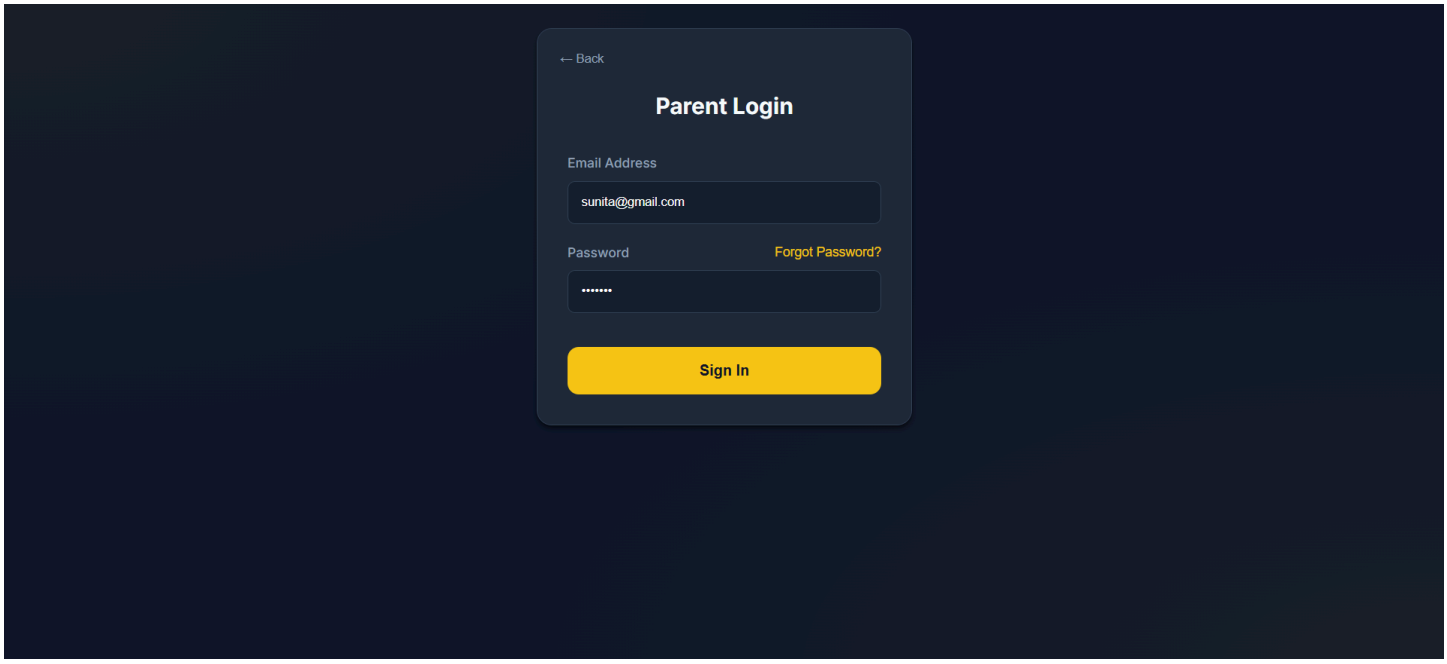
Output Screens & Website Flow

Home Page
The system starts with a role selection screen where the user can choose between Parent, Teacher, or Admin login.

Figure 2

Parent Login Page
Parent enters email and password to access their account. A “Forgot Password” option is also available.

Figure 3



Parent Dashboard Child Selection

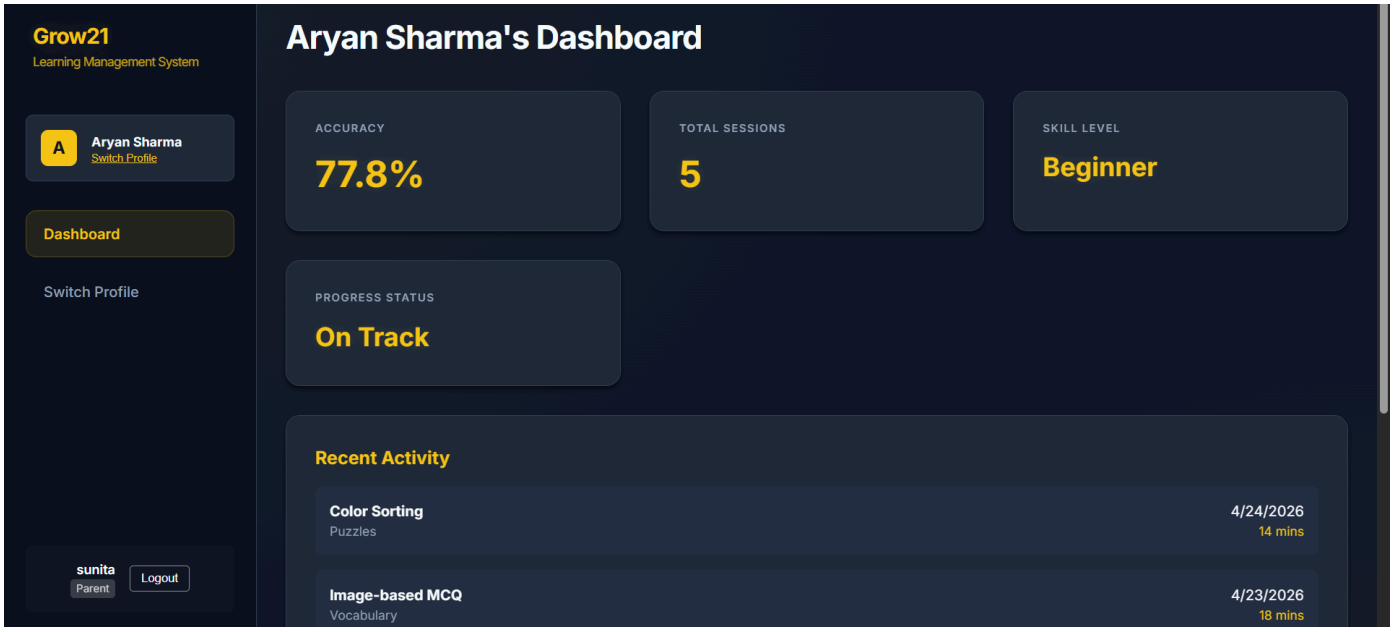
After login, the parent can view all their children and select whose progress they want to see. There is also an option to add a new child.

Figure 4

Parent Dashboard – Child Performance

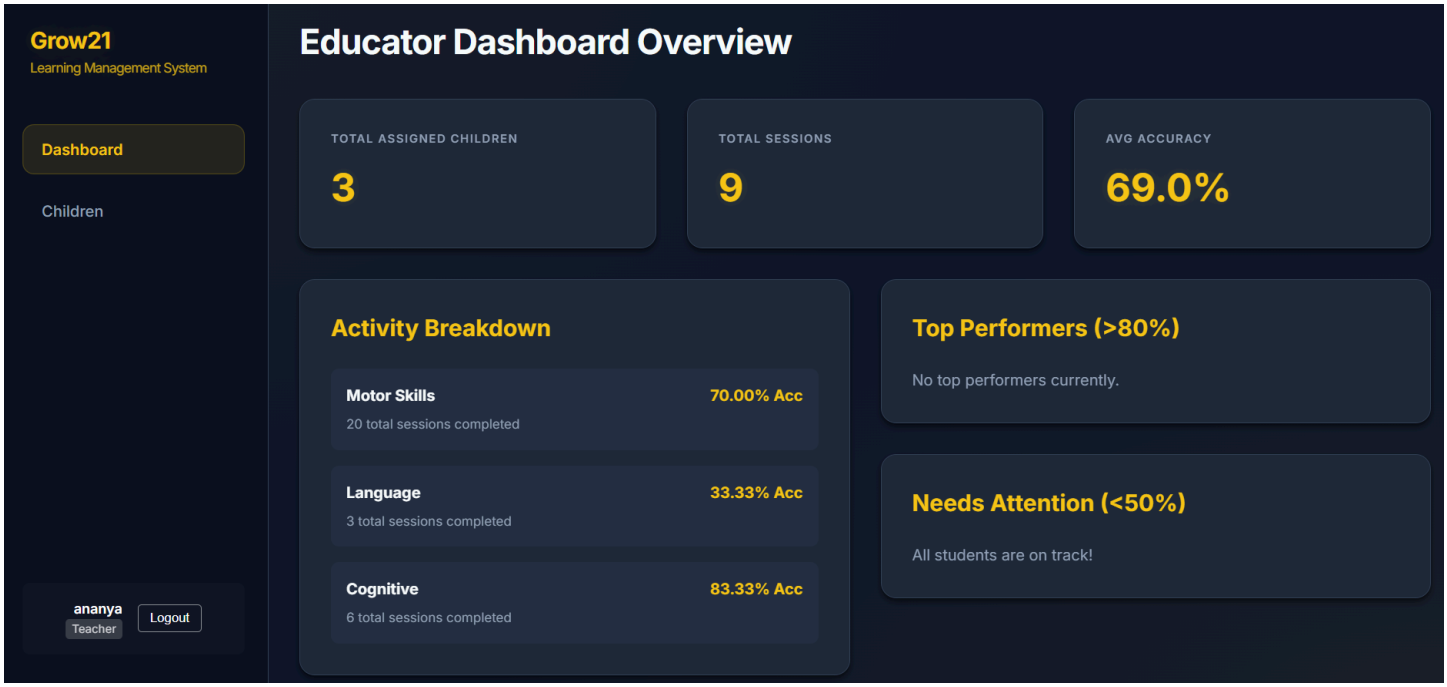
Displays detailed information like accuracy, total sessions, skill level, progress status, and recent activity of the selected child.

Figure 5



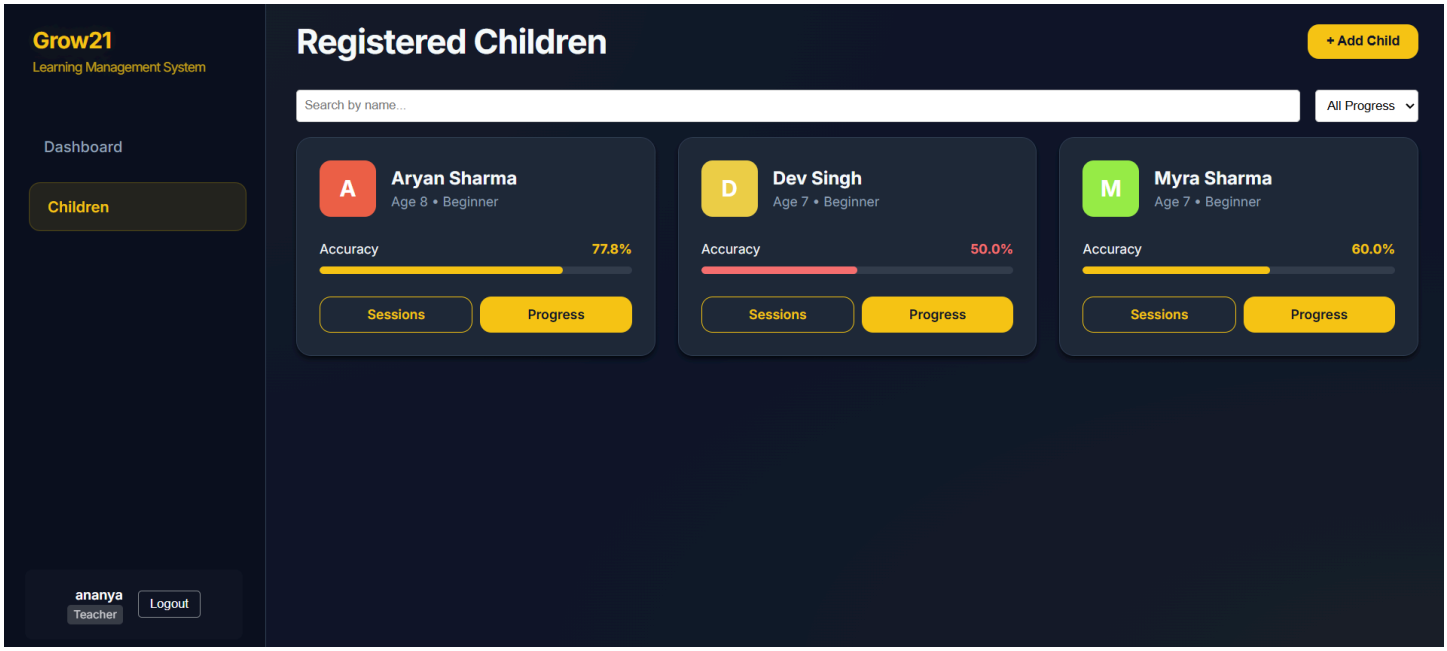
Teacher Dashboard Overview
Shows total assigned children, total sessions conducted, and average accuracy of students.

Figure 6



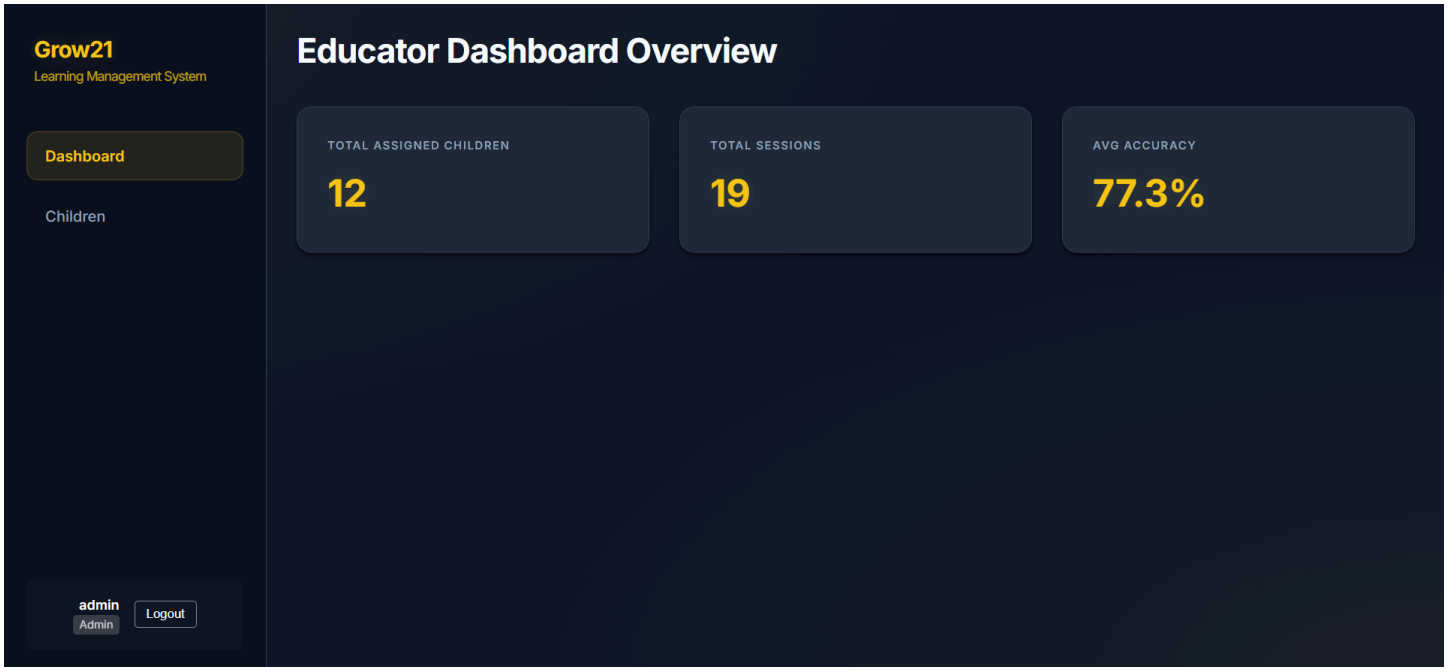
Teacher Registered Children Page
Displays all assigned students with their accuracy levels. Teachers can monitor performance and track progress.

Figure 7



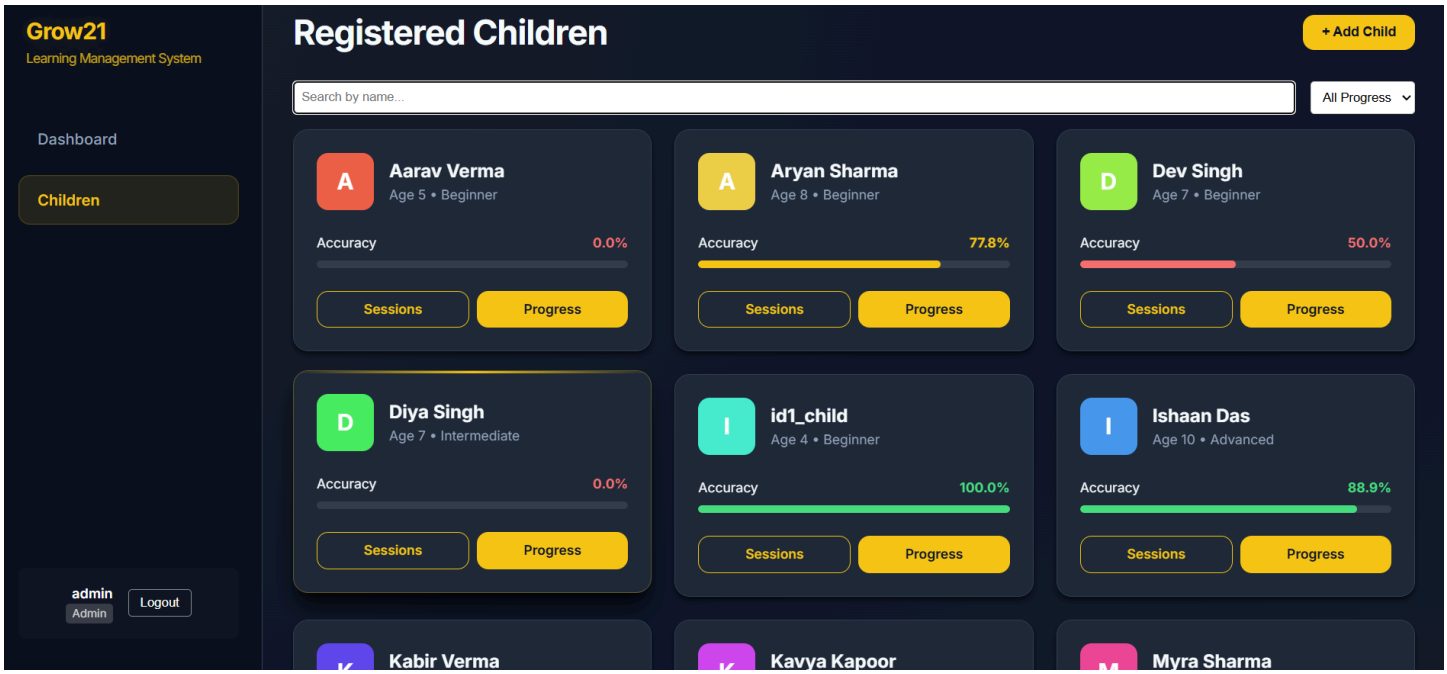
Admin Dashboard Overview
Shows overall system statistics such as total children, sessions, and average accuracy across the platform.

Figure 8



Admin Full Children List
Displays all registered children along with their accuracy levels, enabling complete system monitoring and management..

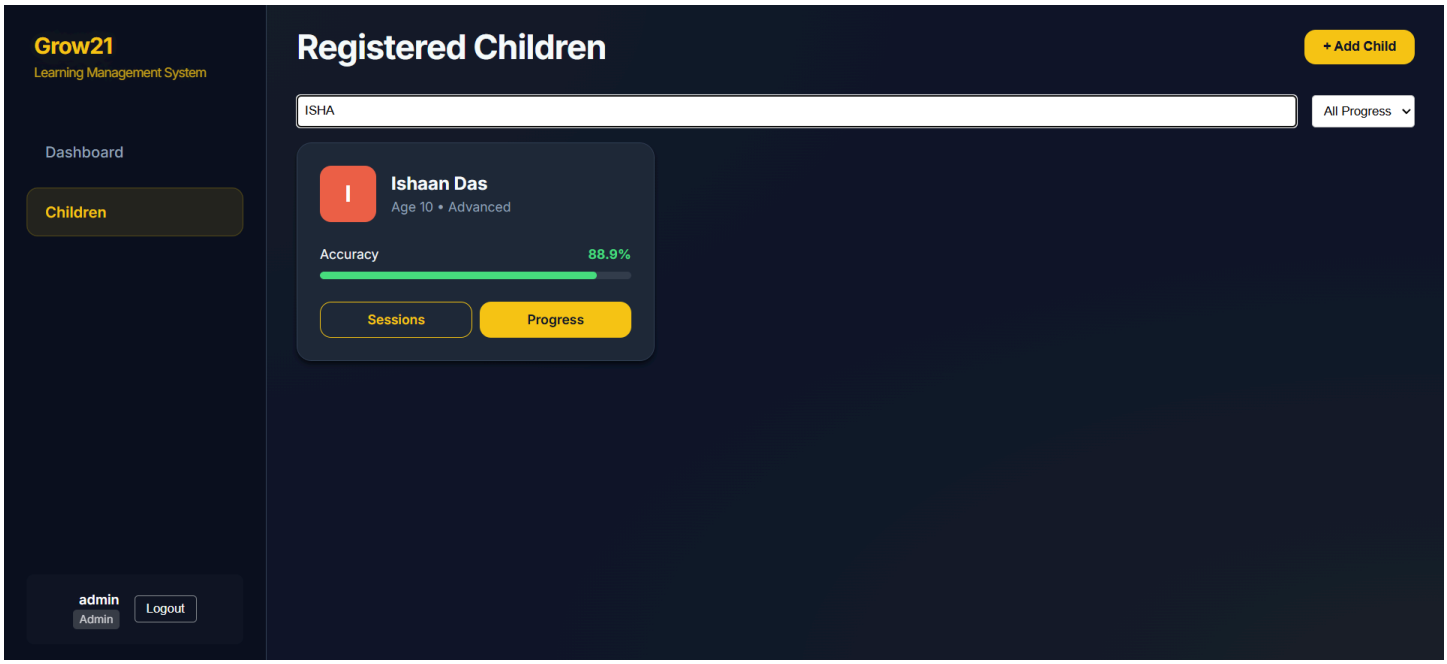
Figure 9



Search Functionality

Users(ie teacher,parents and admin) can search for specific children using the search bar and filter based on progress.

Figure 10



Concluding Remarks

The Grow21 database system successfully models a structured and practical learning platform for children with Trisomy 21. It is designed to store and manage all important data such as user details, child profiles, learning sessions, and performance records in a well-organized manner.

The use of normalization up to Third Normal Form (3NF), along with proper use of primary keys and foreign keys, ensures that the data remains consistent and free from redundancy. Features like views, stored procedures, and queries help in generating meaningful insights such as accuracy and session-based performance. Overall, the system works efficiently in tracking a child's learning progress based on their interaction with the mobile application.

The design is flexible and scalable, which means it can be extended further and integrated into a complete real-world application without major changes.

Societal Relevance

This project is useful in supporting children(with trisomy 21 or downsyndrome) with special learning needs by providing a structured way to track their progress through interactive and engaging activities. It helps parents and teachers get a clear understanding of how a child is performing over time, including their strengths and areas where they may need more attention.

By making progress data easily accessible and understandable, the system encourages better communication between parents and educators. It also supports personalized learning, which is very important for children with developmental conditions like Trisomy 21. Such platforms can contribute to early intervention and more effective learning strategies.

Future Scope

The system can be further improved by adding features such as real-time analytics, graphical dashboards, and AI-based recommendations that suggest activities based on a child's performance.

We are also planning to extend this project for use across different schools. Currently, the system is designed for a single school, and the database contains data specific to that setup. For multiple schools, separate database instances can be maintained for each school, as combining data from different schools in a single database may not be logical and could create complexity in management and access.

In the future, the system can be developed into a full-scale application with proper authentication, notifications, and role-based access. Additional features like progress visualization, feedback systems, and personalized learning paths can further enhance its effectiveness.

Team Details & Contribution

The project was completed collaboratively by all team members, with each person contributing in different areas:

- **Dhruva Jhanjhari** worked on building the basic SQL structure and overall backend–frontend flow. He also created the initial ER diagrams and contributed to the report.
- **Kartikey Bishnoi** enhanced the project by adding advanced SQL queries and providing useful ideas throughout the development. He also helped in preparing the presentation (PPT).
- **Om Dixit** contributed creative ideas for both frontend and backend improvements and also assisted in making the project presentation.
- **Bhavya** focused on connecting the application with the database and significantly improved the frontend interface. She also refined the ER diagram and worked on compiling the report.

Overall, the project was a team effort with everyone actively contributing to both technical and documentation parts.

Use of AI Tools

In this project, AI tools were used to a limited extent mainly for assistance and guidance. Overall, approx. 30-40% of the project involved the use of AI, primarily in areas such as connecting the application components and improving the user interface.

For the core part of the project, which is the SQL database design and implementation, minimal AI assistance was used. The database schema, relationships, normalization, and queries were developed and understood by the team.